## CHAPTER – XI

## GENERICS IN COLLECTIONS FRAMEWORK

### GENERICS IN JAVA:

- Generics allow Java classes, methods, and collections to work with type safety by specifying the data type at compile time.
- **SYNTAX:**

```
ArrayList<String> list = new ArrayList<>();
```

### TYPES:

- Generic class
- Generic methods

### CUSTOM OBJECT WITH GENERICS:

- A custom object is an object created from a user-defined class.
- A custom object is an object created from a class that YOU define, not a built-in class like String or Integer.

### EXAMPLE:

```
class Student<T> {
   T data;
   Student(T data) {
      this.data = data;
   }
   void display() {
      System.out.println(data);
```

```
    }
}
public class GenericExample {
    public static void main(String[] args) {
        Student<String> s1 = new Student<>("Shalini");
        Student<Integer> s2 = new Student<>(101);
        s1.display();
        s2.display();
    }
}
```

**SORTING IN COLLECTIONS:**

- Sorting in collections means arranging elements in a specific order (ascending or descending).
- In Java, collections can be sorted using:
  - Comparable (natural sorting)
  - Comparator (custom sorting logic)

**COMPARABLE:**

- Comparable is an interface used to define the natural sorting order of objects.
- A class implements Comparable and overrides the compareTo() method.

**EXAMPLE:**

```
import java.util.*;
class Student implements Comparable<Student> {
    int id;
    Student(int id) {
        this.id = id;
```

```
    }
    public int compareTo(Student s) {
        return this.id - s.id;
    }
}
public class ComparableDemo {
    public static void main(String[] args) {
        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student(3));
        list.add(new Student(1));
        list.add(new Student(2));
        Collections.sort(list);
        for (Student s : list) {
            System.out.println(s.id);
        }
    }
}
```

**COMPARATOR:**

- Comparator is an interface used to define custom or multiple sorting orders.
- Sorting logic is written outside the class.

**EXAMPLE:**

```
import java.util.*;
class Student implements Comparable<Student> {
    int id;
    Student(int id) {
```

```
        this.id = id;
    }
    public int compareTo(Student s) {
        return this.id - s.id;
    }
}
public class CompareToExample {
    public static void main(String[] args) {
        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student(3));
        list.add(new Student(1));
        list.add(new Student(2));
        Collections.sort(list);
        for (Student s : list) {
            System.out.println(s.id);
        }
    }
}
```